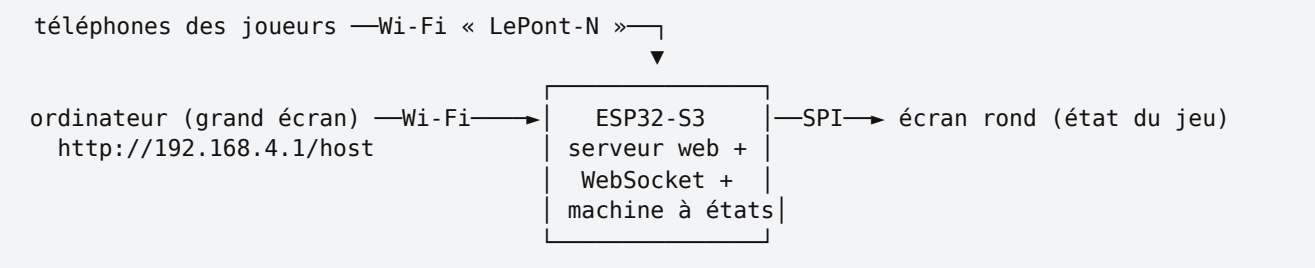
Le Pont France – Vietnam · Cầu Pháp – Việt

Version V2 « chantier » (29 octobre 2026) : nouveau gameplay à deux rives (voir ci-dessous), écran rond GC9A01 et tactile CST816S (`DISPLAY_TYPE 2`, `TOUCH_ENABLE 1`), croquis de test et démo d'animation dans `tests_materiel/`, images via `assets/png2assets.py`. L'écran de l'objet est dessiné dans un canvas à partir des images de `assets/` (décor, pont en arc découpé en 10 tranches, grue, chef de chantier Cầu, dragon, scène de victoire) avec des animations non bloquantes (`displayTick()`). Archive : `LePont_v2.zip`. Le simulateur `test/mock_server.py` rejoue les mêmes règles en Python, sans carte ; `tests_materiel/Demo_Animation` fait tourner les écrans du jeu sans Wi-Fi.

Un jeu de questions-réponses interculturel posé au centre de la table : un petit objet (ESP32-S3 + écran rond) crée son propre Wi-Fi, les joueurs répondent depuis leur smartphone, un grand écran montre le pont qui se construit à chaque bonne réponse collective. Né à l'atelier **PFIEV × DUT Maker Innovation Space** (Da Nang, 2026), conçu pour être reproduit et adapté n'importe où : un autre pays, une autre histoire partagée, une autre langue.

Licence : **MIT** pour le code, **CC BY-SA 4.0** pour les questions et la documentation.

Comment ça marche



1. L'objet crée un point d'accès Wi-Fi avec portail captif ; les téléphones ouvrent `http://192.168.4.1`.
2. Le grand écran (`/host`) est aussi la télécommande : Lancer · Suivante · Révéler · Reset. Avec l'écran tactile, un tap sur l'objet fait la même chose.
3. `/admin` : coller de nouvelles questions sans reflasher ; `/api/state` : état de l'objet.

Les règles du chantier (V2)

- **Deux rives.** À l'arrivée, chaque joueur choisit la rive France ou la rive Vietnam (ou « au hasard » : l'objet équilibre). Les deux rives jouent les mêmes questions en même temps, 20 s par question.
- **Chaque rive construit sa moitié.** Quand la majorité d'une rive a juste, elle pose une travée de son côté : 4 travées par rive, de la berge vers le centre.
- **La rencontre.** Quand les deux rives ont leurs 4 travées, une question « double » pose la travée centrale si les deux rives réussissent ensemble. Sinon on retente, avec une option en moins à chaque fois.
- **Le pont se ferme toujours.** Une mauvaise réponse ne détruit rien : la travée reste suspendue à la grue, la question revient trois tours plus tard. S'il ne reste plus de questions, « le dernier coup de main » ferme le pont.
- **Le chantier s'adapte, par rive.** Chaque question a un niveau (1 facile, 2 moyen, 3 difficile ; colonne `niveau`). Deux réussites de suite : niveau au-dessus ; deux échecs : niveau en dessous. Au niveau 1, ou après trois échecs sur la même travée, la question est **assistée** : deux options au lieu de quatre, 30 s, et un indice du chef de chantier (champ `hfr` de la question, sinon la première phrase de la révélation).
- **Estimation.** Pour une question à cible numérique (`num`), chaque joueur tape un nombre ; une rive réussit si sa médiane est à ±40 % de la cible (±5 ans pour une année).
- **La fiche de chantier.** Pas de gagnant ni de perdant : à la fin, le nombre de questions, la durée, les indices utilisés, la question à laquelle chaque rive a fini, et les points de vitesse (100 + bonus de rapidité par bonne réponse individuelle). C'est le score à battre par les autres tables.
- Rive vide (un seul joueur, par exemple) : elle suit automatiquement le résultat de l'autre.
- Petits groupes (1 à 9 joueurs) : « Lancer » reste grisé tant qu'aucun joueur n'est connecté ; au lancement, si une rive est vide alors que l'autre a 2 joueurs ou plus, un joueur change de rive (son téléphone l'annonce). Avec un seul joueur, il construit tout le pont.

- Le moniteur série affiche une ligne `[etat]` toutes les 10 s : appareils sur le Wi-Fi, pages ouvertes, joueurs, mémoire libre (10 appareils au maximum, ordinateur compris).

Matériel (par objet)

- 1 × ESP32-S3 DevKitC-1 **N16R8** (16 Mo flash, 8 Mo PSRAM)
- 1 × écran SPI 240×240 : ST7789 1,3" (carré) ou GC9A01 1,28" (rond, tactile)
- 8 fils Dupont femelle-femelle (12 avec le tactile), 1 câble USB-C de données, 1 powerbank
- Un boîtier modélisé et imprimé en 3D par le groupe

Câblage écran → ESP32-S3 : VCC→3V3 · GND→GND · SCL→12 · SDA→11 · CS→10 · DC→9 · RES→8 · BLK→7

Fichiers

Fichier	Rôle	Qui le modifie
`LePont/LePont.ino`	réseau, serveur, machine à états, scores	les curieux
`LePont/config.h`	numéro de groupe, écran, broches, équipes	**tout le monde, en premier**
`LePont/questions.h`	la base de questions (JSON)	**tout le monde**
`LePont/display.h`	l'écran de l'objet (reçoit une `BridgeView` : travées par rive, grue, rencontre)	**votre terrain de jeu**
`LePont/touch.h`	le tactile CST816S (Niveau 2, `TOUCH_ENABLE 1`) : tap = suite, appui long = nouvelle partie	Tutoriel 2
`tests_materiel/Test_Ecran`, `Test_Tactile`	croquis de test du câblage, à téléverser avant le jeu	Tutoriel 2
`tests_materiel/Demo_Animation` + `assets/`	moteur d'animation (canvas 30 i/s) et convertisseur d'images PNG → `assets.h`	V2 graphique
`test/mock_server.py`, `test/e2e.py`	simulateur des règles en Python et test automatique des pages dans un navigateur	sans carte
`LePont/web_player.h`	page des téléphones : la scène Paris – Hội An, le pont qui se construit, le petit dragon, les sons	soir 2
`LePont/web_host.h`	page grand écran (vidéoprojecteur) : la même scène en grand, la question, la révélation ; touches → ou Espace = suite, R = révéler, F = plein écran	soir 2
`LePont/web_admin.h`	page de modification des questions	rarement
`web/*.html`	sources des pages ; `gen_web_h.py` les régénère en `.h`	si vous préférez éditer du .html
`web/a/` → `LePont/web_assets.h` + `web_assets.cpp`	images et polices du téléphone (illustrations, Be Vietnam Pro, Playfair Display), servies sous `/a/…` et gardées en cache ; `gen_web_h.py` crée `web_assets.h` et `web_assets.cpp` (compilé une fois puis réutilisé : les téléversements suivants restent rapides)	ne pas modifier à la main
`test/`	serveur de simulation (Python) pour tester les pages sans ESP32	optionnel

Installation : voir **GUIDE_INSTALLATION.md**.

Protocole WebSocket (JSON) — pour vibe coder de nouvelles manches

Sens	Message	Contenu
téléphone → objet	`join`	`name`, `team` (0 France, 1 Vietnam, -1 au hasard)

Sens	Message	Contenu
téléphone → objet	`answer`	`a` (index de l'option affichée) ou `v` (nombre, estimation)
grand écran → objet	`hello`, `host`	`cmd` : `start` · `next` · `reveal` · `reset`
objet → tous	`config`	`ssid`, `sec`, `per` (travées par rive), `teams`
objet → tous	`lobby`	`players`, `bridge`
objet → tous	`question`	`i`, `sec`, `lv`, `dbl`, `cat`, `fr`, `vi`, `o` (mélangées) ou `num`, `hide[rive]`, `assist[rive]`, `hint`, `bridge`
objet → tous	`tick`	`left`, `answers[rive]`
objet → tous	`reveal`	`r`, `pfr`, `pvi`, `res[rive]`, `nc[rive]`, `na[rive]`, `center`, `target`/`med` (estimation), `bridge`
objet → tous	`end`	`qs`, `dur`, `hints`, `doneAt[rive]`, `scores`, `bridge`
objet → un joueur	`joined` (`team` attribuée), `you` (`ok`, `pts`), `full`	

`bridge` = `{side:[4,4], pending:[false,true], center:false, per:4, level:[2,1]}` : tout ce qu'il faut pour dessiner le chantier.

Les trois soirées

Soir 1 — ça marche. Installer, flasher la base, jouer une partie avec les 10 questions du groupe, lire `LePont.ino` ensemble, câbler l'écran et passer `DISPLAY_TYPE` à 1.

Soir 2 — c'est le vôtre. Vibe coding : une nouvelle manche (estimation, mot caché, vrai/faux), un nouvel écran dans `display.h`, de nouvelles règles de points. Modéliser le boîtier.

Soir 3 — c'est fini et partagé. Imprimer, assembler, jouer, documenter, démontrer.

Prompts de départ pour l'IA (copier-coller avec le fichier concerné)

- « Voici `display.h`. Remplace l'écran de question par un cadran : anneau de compte à rebours qui change de couleur, numéro de question au centre, et une bordure aux couleurs de l'équipe qui mène. Garde les mêmes six fonctions et la bibliothèque Arduino_GFX. »
- « Voici `LePont.ino` et `questions.h`. Ajoute un type de question `num` (estimation) : les joueurs tapent un nombre, le plus proche de `num` gagne 150 points, les autres 0. Ajoute le champ de saisie dans `web_player.h` et l'affichage des estimations dans `web_host.h`. »
- « Voici `web_host.h`. Dessine le pont comme une arche entre deux rives avec un dragon qui s'allume quand toutes les travées sont posées. SVG pur, pas de bibliothèque externe. »
- « Voici `config.h` et `LePont.ino`. Passe à 4 équipes avec un code couleur, et affiche le classement des équipes à la fin. »

Documentation open source attendue de chaque groupe

Un dossier avec : le code modifié, les fichiers 3D (STL + source), une photo du câblage, vos questions (FR/VI), une vidéo de 30 s du jeu en action, et un `README` qui explique comment reproduire votre objet et ce que vous changeriez pour un autre pays. Publiez-le sous licence libre.